

Limited Direct Execution (LDT)

Ch6

Sk Subidh Ali
IIT Bhilai

Running a process

Kernel

Program

-
1. Create entry in kernel process list
 2. Allocate memory for the program
 3. Load the program into memory
 4. Set up stack with argv/argc
 5. Clear CPU registers
 6. Call main() of the program
 7. CPU executes main()
 8. Execution return from main()
 9. Free memory of the process
 10. Remove from process list

Challenge: When you allow a program to run in CPU it can do anything

- Corrupt even kernel data/code

Running a process

Solution

Kernel

Program

1. Create entry in kernel process list
2. Allocate memory for the program
3. Load the program into memory
4. Set up stack with argc/argv
5. Clear CPU registers
6. (This part of the code runs in CPU's kernel/privilege model)
9. Free memory of the process
10. Remove from process list

This part of the code runs in CPU's user/unprivilege model

7. CPU executes main()
8. Execution return from main()

A process can get resources or operating system services through system calls

Trap handling

```
printf("%s", str);
```

```
printf("hi\n");
```

libc invocation

```
write(1, "hi\n", 3)
```

User
space

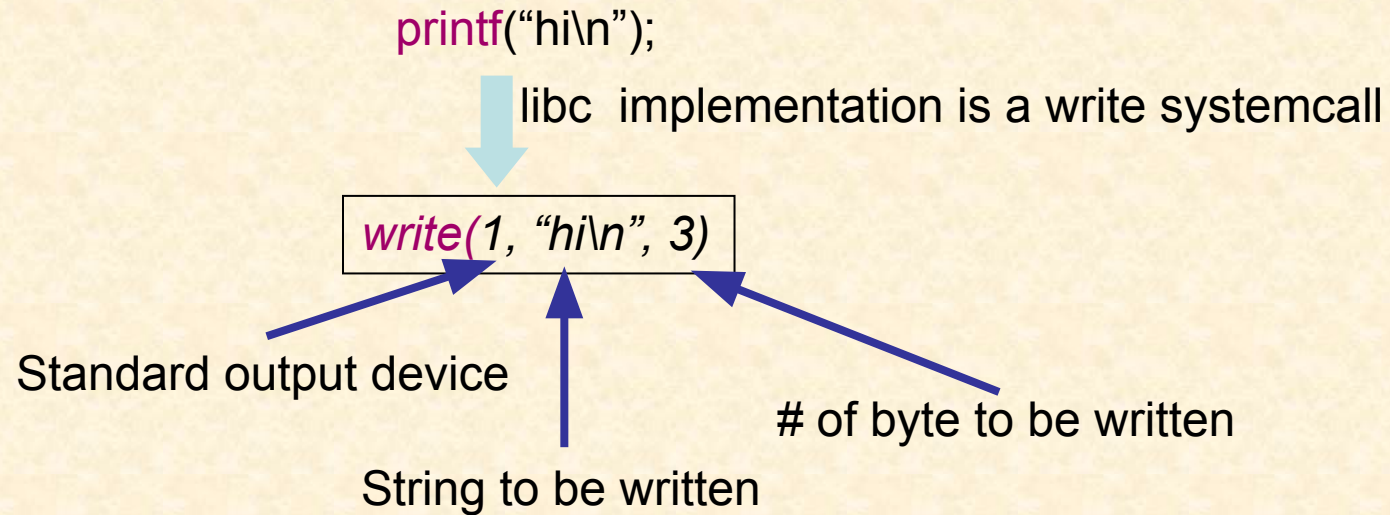
libc invocation
trap

Kernel
space

Trap
Handler

Implementatio
n
of
write syscall

Trap handling:



Trap/interrupt handling

User mode

Kernel mode

CPU:

1. changes privilege level to kernel mode.
2. saves interrupt/trap number and processor status: PC, Stack, and other registers
3. Jump to interrupt/trap handler (a kernel function `trap()`)

`trap()`

1. Load the trap number
2. Execute the corresponding trap handler
3. Restores the process state
4. Change CPU privilege to user model

User process resumes:
Start executing after the write system call

Execution transfer in function call

How the execution returns to user program after system call execution, which is kernel function?



Typical function call

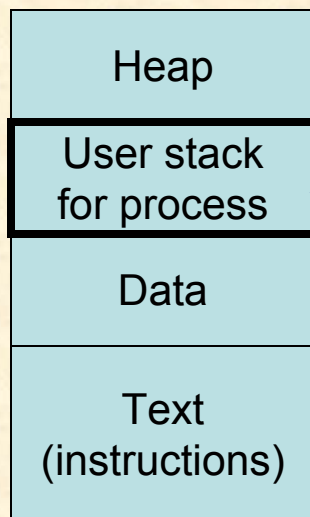
```
#include<stdio.h>
```

```
int main()  
{  
  Int a=5,b=6,c;  
  c=sum(a,b);  
  c=c+1;  
}
```

```
int sum(int x, int y)  
{  
  return(x+y);  
}
```

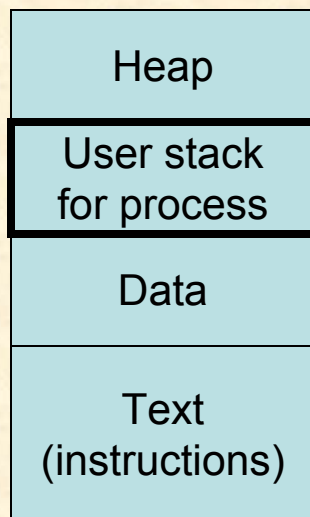
While executing
sum(a,b), PC and
local variables are
pushed into stack.

Execution jump
to sum()



Execution transfer in function call

C function call



```
#include<stdio.h>
```

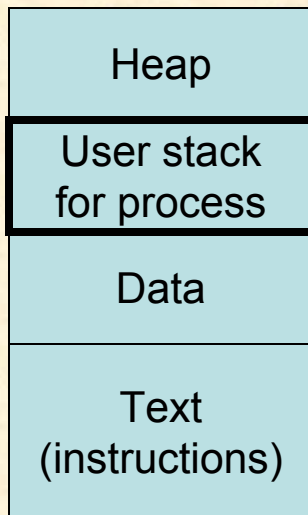
```
int main()  
{  
    Int a=5,b=6,c;  
    c=sum(a,b);  
    c=c+1;  
}
```

```
int sum(int x, int y)  
{  
    return(x+y);  
}
```

While executing return(a,b), PC and local variables of main are popped from stack. CPU execute from PC.

Execution transfer in Trap/interrupt handling

Printf is implemented with write() system call in c library.
Again **write()** is using **int n** (n is interrupt no) instruction in
Intel and **ecall** instruction in RISC V



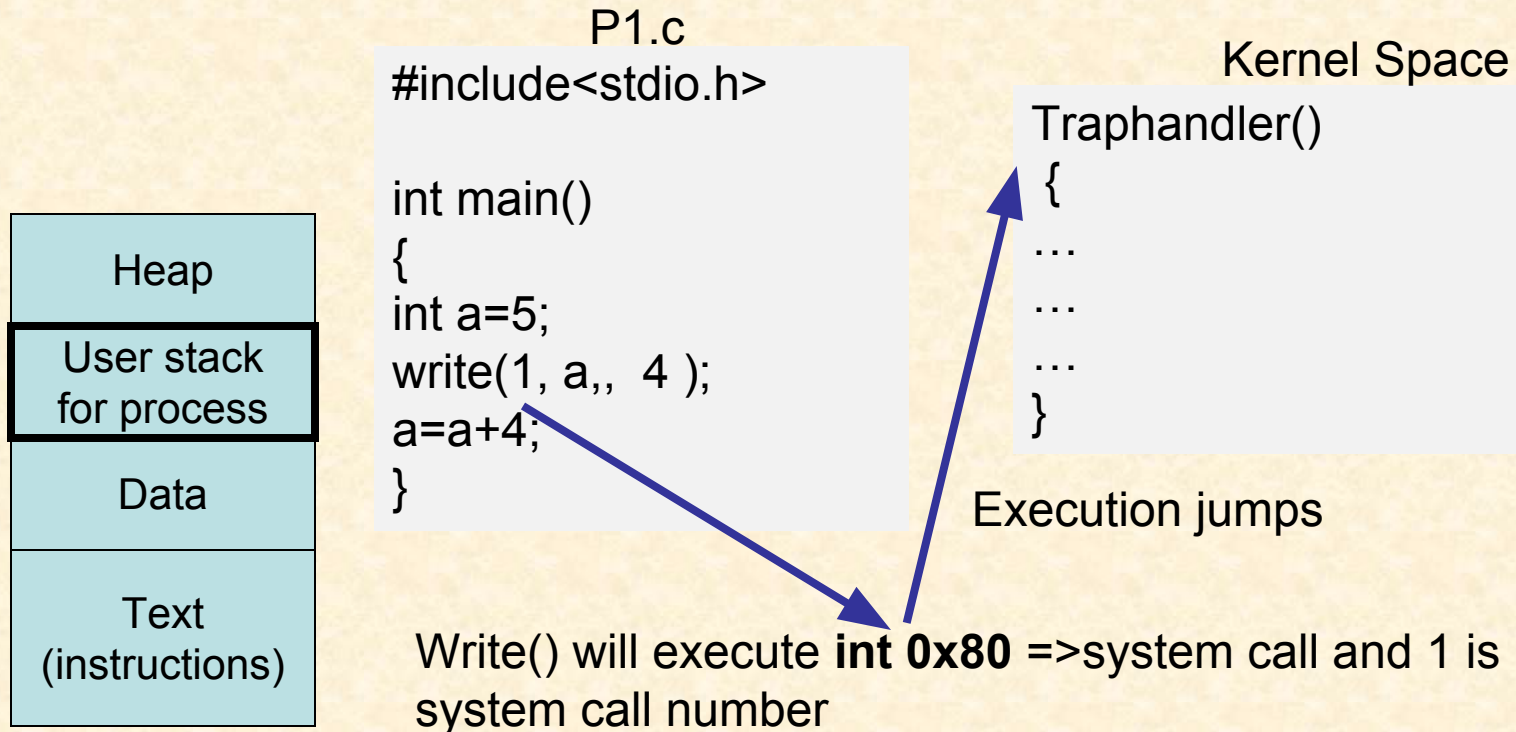
```
P1.c
#include<stdio.h>

int main()
{
    int a=5;
    printf("%d",a);
    a=a+4;
}
```

Execution transfer in Trap/interrupt handling

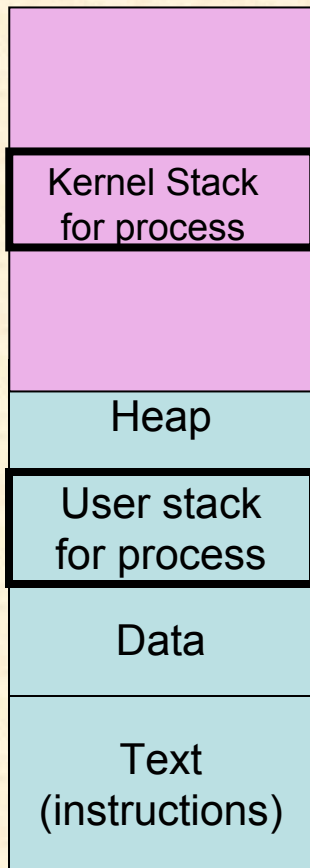
Printf is implemented with write() system call in c library.
Again **write()** is using **int n** (n is interrupt no) instruction in
Intel and **ecall** instruction in RISC V

If just replace printf with its library implementation



How execution returns from system call

Now if we save return address in user process stack it will never return as kernel has no clue which process was running. It could be any ready process

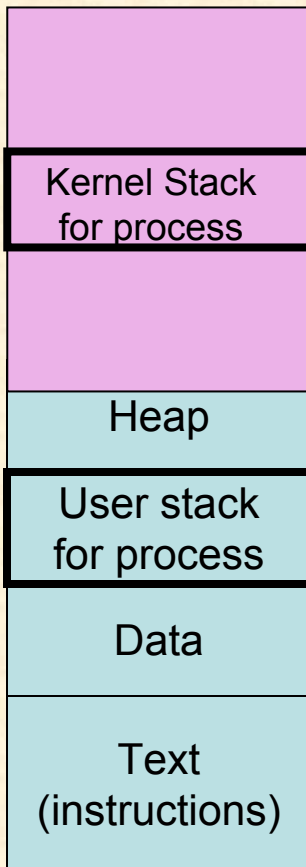


Solution: Two stacks are maintained in each process

1. Process stack: used for regular function calls in the process
 - Accessible in user mode
2. Kernel stack: used for kernel function such as system calls
 - Accessible in kernel/supervisor/machine mode

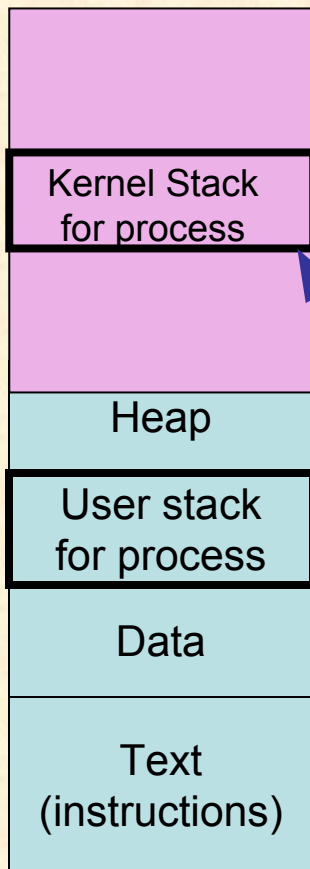
During system call:

1. Context of the process is pushed in kernel stack
2. CPU privileged is set to kernel mode



During system call:

1. CPU privilege is set to kernel mode
2. Context of the process is pushed in kernel stack
3. Execution jump to trap handler
4. Trap handler:
 - Executes system call
 - Set CPU privilege to user mode
 - Load the context of user process to CPU. It includes next address to PC
 - Process start executing in user mode



PC and
Other
registers

```
#include<stdio.h>
```

```
int main()  
{  
  int a=5;  
  write(1, a, 4 );  
  a=a+4;  
}
```

```
Traphandler()  
{  
  ...  
  ...  
  ...  
}
```

What happens when process completes execution

```
#include<stdio.h>
```

```
int main()  
{  
  int a=5;  
  write(1, a,, 4 );  
  a=a+4;  
  exit()  
}
```

It follows step 1-3 of system call execution

- Traphandler for exit:
 - Will release most of the resources of the user process and then call kernel scheduler
- Scheduler then scan for ready process in the list
 - Load the context of the process and run the process in user mode

System Call vs Procedure Call

System Call	Procedure Call
Uses a TRAP instruction (such as int 0x80 or ecall)	Uses a CALL instruction
System shifts from user space to kernel space	Stays in user space (or kernel space) ... no shift
TRAP always jumps to a fixed address (depending on the architecture)	Re-locatable address

ASIDE: KEY CPU VIRTUALIZATION TERMS (MECHANISMS)

- The CPU should support at least two modes of execution: a restricted **user mode** and a privileged (non-restricted) **kernel mode**.
- Typical user applications run in user mode, and use a **system call** to **trap** into the kernel to request operating system services.
- The trap instruction saves register state carefully, changes the hardware status to kernel mode, and jumps into the OS to a pre-specified destination: the **trap table**.
- When the OS finishes servicing a system call, it returns to the user program via another special **return-from-trap** instruction, which reduces privilege and returns control to the instruction after the trap that jumped into the OS.
- The trap tables must be set up by the OS at boot time, and make sure that they cannot be readily modified by user programs. All of this is part of the **limited direct execution** protocol which runs programs efficiently but without loss of OS control.
- Once a program is running, the OS must use hardware mechanisms to ensure the user program does not run forever, namely the **timer interrupt**. This approach is a **non-cooperative** approach to CPU scheduling.
- Sometimes the OS, during a timer interrupt or system call, might wish to switch from running the current process to a different one, a low-level technique known as a **context switch**.

Next class: CPU Scheduling : Ch 7 Remzi